Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 9: **Pengolahan Data: Import, Export, dan Queue**

Dari pipeline ETL, dataset besar, dan memory, sampai pekerjaan berat yang dititipkan ke background

Yang Akan Kamu Pelajari

1. Menjelaskan pola **ETL** dan **batch processing**, dan memakainya untuk memproses data dalam jumlah besar
2. Menyusun laporan yang merekap data dalam rentang tanggal, dan memilih teknik yang tepat untuk **dataset besar**
3. Memahami kenapa **memory** bisa habis, dan memilih cara membaca data yang hemat memory
4. Memilih format **CSV, Excel, atau PDF** sesuai kebutuhan, dan mengenal library untuk membuatnya
5. Mengenali kapan sebuah pekerjaan sebaiknya dititipkan ke **queue**, dan membuatnya tahan gagal

> Slide ini membahas konsep. Menerapkan laporan, export/import, dan queue pada Simple POS dikerjakan di jobsheet praktikum.

Peta Pertemuan: Seperti Dapur Restoran

Data mentah → **ETL + Batch** → **Laporan** → **CSV / Excel / PDF** → **Queue**

> Bayangkan dapur restoran. Bahan mentah masuk (**extract**), dimasak jadi hidangan (**transform**), lalu disajikan ke meja (**load**). Kalau pesanan membludak, dapur tidak menyuruh tamu berdiri di depan kompor: tamu diberi nomor meja, lalu hidangan diantar belakangan. Itulah gambaran besar pertemuan ini.

Bagian 1

ETL dan Batch Processing

Empat Istilah Dasar

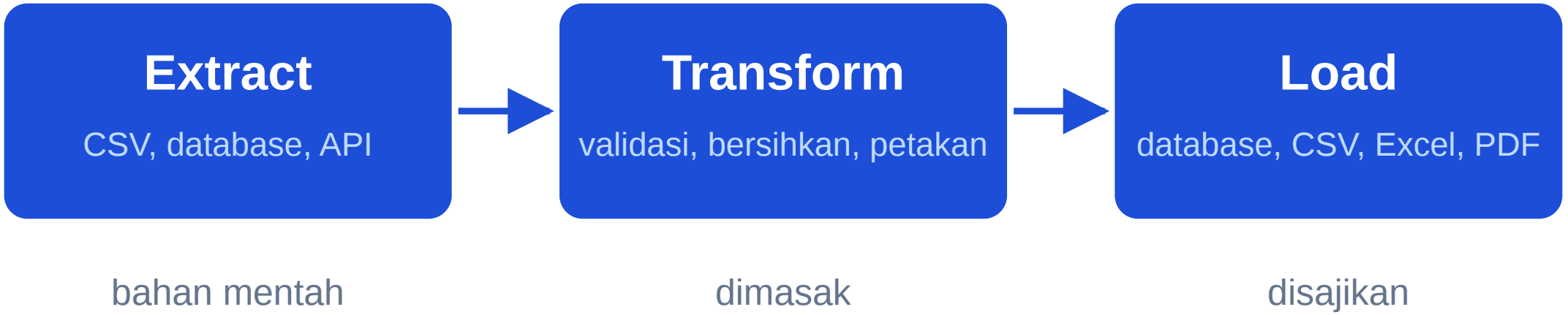
ETL (Extract, Transform, Load): pola umum mengambil data mentah, mengubah bentuknya, lalu menyimpan atau menyajikannya dalam bentuk baru.

Batch Processing: memproses banyak data sekaligus dalam kelompok, bukan satu per satu.

Queue: mekanisme menitipkan pekerjaan berat agar dikerjakan di background oleh proses terpisah, bukan langsung selama request pengguna berlangsung.

Import/Export: membawa data masuk dari format eksternal (CSV, Excel) ke database, atau mengeluarkannya ke format eksternal (CSV, Excel, PDF) untuk dibaca di luar aplikasi.

ETL: Tiga Tahap yang Selalu Muncul



Hampir semua pengolahan data mengikuti urutan ini, baik import maupun export. Memisahkan tahapnya membuat tiap bagian mudah diuji dan diganti: sumber datanya boleh berubah, aturan validasinya boleh berubah, tujuannya boleh berubah.

Import dan Export Adalah ETL Dua Arah

	Extract	Transform	Load
Import	baca file CSV / Excel	validasi tiap row, petakan column	simpan ke database
Export	query database	pilih column, format tanggal dan angka	tulis CSV / Excel / PDF

> Pertanyaan yang berguna untuk setiap fitur import atau export: dari mana datanya (extract), apa yang harus dibersihkan atau diubah (transform), dan ke mana hasilnya (load)?

Batch: Satu per Satu vs Satu Truk

Kurir yang mengantar satu paket per perjalanan akan kalah efisien dari kurir yang membawa satu truk penuh. Hal yang sama berlaku untuk database: setiap query punya ongkos perjalanan (round trip) ke database.

```php
// 10.000 row = 10.000 query
foreach ($rows as $row) {
    Article::create($row);
}

// 10.000 row = 20 query (batch 500 row)
foreach (array_chunk($rows, 500) as $batch) {
    Article::insert($batch);
}
```

> `insert()` massal tidak mengisi `created_at` / `updated_at` otomatis dan tidak menjalankan model event. Isi timestamp-nya sendiri kalau dibutuhkan.

Ukuran Batch dan Membaca per Batch

Batch terlalu kecil

- Terlalu banyak round trip
- Lambat, seperti kurir motor

Batch terlalu besar

- Memory membengkak
- Satu kegagalan menggagalkan banyak row

Membaca per batch

```
Article::where('status', 'draft')
    ->chunkById(500, function ($articles) {
        foreach ($articles as $a) {
            // proses tiap row
        }
    });
```

> Titik awal yang wajar: ratusan sampai seribu row per batch, lalu ukur. Bungkus tiap batch dengan `DB::transaction()` kalau satu batch harus berhasil atau gagal seluruhnya.

Idempotent: Aman Dijalankan Berkali-kali

Bagaimana kalau file import yang sama di-upload dua kali? Tanpa penjagaan, semua row tercatat dobel. Bayangkan daftar hadir: kalau namamu sudah ada, petugas memperbarui barisnya, bukan menulis nama yang sama lagi.

```
Article::updateOrCreate(
    ['slug' => $row['slug']],     // kunci pencarian
    ['title' => $row['title']],   // nilai yang diperbarui
);

// versi batch: satu query untuk banyak row
Article::upsert($batch, ['slug'], ['title']);
```

`upsert()` mensyaratkan **unique index** pada column kuncinya. Tanpa itu, database tidak tahu mana row yang dianggap sama.

Bagian 2

Laporan dan Dataset Besar

Daftar vs Laporan: Tumpukan Struk vs Rekap Akhir Bulan

Halaman daftar itu seperti tumpukan struk di laci kasir: semua ada, tapi kamu yang harus menghitungnya sendiri. Halaman laporan itu seperti rekap akhir bulan: sudah dijumlahkan dan siap dibaca. Pertanyaannya lebih sempit tapi lebih berat: "berapa total X pada rentang waktu tertentu".

```
$totalKata = Article::whereBetween('published_at', [
    $from, $to,
])->sum('word_count');
```

`whereBetween` memfilter row yang tanggalnya berada di antara dua batas, lalu `sum()` menjumlahkan satu column dari hasil filter itu.

Jebakan Umum: Batas Hari yang Terpotong

> Filter "1 Agustus sampai 31 Agustus" yang ditulis apa adanya berarti "sampai jam 00:00:00 tanggal 31". Seluruh transaksi yang terjadi sepanjang tanggal 31 ikut hilang, seperti toko yang menghitung omzet bulan Agustus tapi berhenti saat baru buka pintu di hari terakhir.

```
$from = $request->date('from')->startOfDay();
$to = $request->date('to')->endOfDay();
```

`startOfDay()` mengubah tanggal awal menjadi jam 00:00:00, dan `endOfDay()` mengubah tanggal akhir menjadi jam 23:59:59, supaya seluruh hari terakhir ikut terhitung.

Jebakan Batas Hari, Digambar di Garis Waktu



Hitung di Database, Bukan di PHP

Bayangkan menanyakan saldo rekening. Kamu bisa meminta bank mengirim seluruh mutasi lalu menjumlahkannya sendiri, atau cukup meminta bank menyebutkan saldonya. Database adalah banknya.

```php
// berat: semua row dimuat ke memory, dijumlahkan di PHP
$total = Article::all()->sum('word_count');

// ringan: database yang menjumlahkan
$total = Article::sum('word_count');

// rekap per author: 5 author dengan kata terbanyak
Article::selectRaw('author_id, COUNT(*) AS jumlah, SUM(word_count) AS total_kata')
    ->groupBy('author_id')
    ->orderByDesc('total_kata')
    ->limit(5)
    ->get();
```

Total yang Sudah Tersimpan, Tidak Dihitung Ulang

Pertemuan 6 menegaskan: nilai turunan seperti total tidak boleh dipercaya kalau datang dari input klien, dan harus dihitung ulang di server saat data dibuat. Begitu nilai itu tersimpan dengan benar, laporan boleh memakainya apa adanya. Ini seperti total di nota yang sudah dicetak: saat rekap, kamu cukup menjumlahkan angka di nota, bukan mengecek ulang tiap item di dalamnya.

```
// word_count dihitung sekali saat artikel disimpan
$article->word_count = str_word_count($article->body);

// laporan cukup menjumlahkan column yang sudah tersimpan
$totalKata = Article::whereBetween('published_at', [$from, $to])
    ->sum('word_count');
```

> Menghitung ulang nilai dari data mentah (misalnya menghitung kata dari seluruh isi artikel) setiap kali laporan dibuka membebani server tanpa alasan: angkanya sudah benar sejak artikel itu disimpan.

Perkakas untuk Dataset Besar

Teknik	Dipakai saat	Catatan
`paginate()`	menampilkan daftar ke pengguna	satu halaman per request
`chunkById()`	memproses atau meng-update banyak row	memory per chunk, aman saat row berubah
`lazyById()`	membaca banyak row seperti `foreach`	memory datar, kode sederhana
`cursor()`	membaca satu query panjang	praktis, tapi kurang aman kalau data berubah
`select([...])`	hanya butuh sebagian column	tiap model jadi lebih kecil
`with()`	memakai relasi di dalam loop	menghindari N+1 query
index	filter atau urut pada column tertentu	query tidak scan seluruh tabel

Index: Daftar Isi untuk Tabel

Filter tanggal pada laporan akan dijalankan terus-menerus. Tanpa index pada column itu, database harus membaca seluruh tabel row demi row, seperti mencari satu kata di kamus dengan membaca halaman demi halaman. Dengan index, database melompat langsung ke halaman yang tepat lewat daftar isi.

```
Schema::table('articles', function (Blueprint $table) {
    $table->index('published_at');
});
```

```
EXPLAIN SELECT * FROM articles WHERE published_at BETWEEN '2026-08-01' AND '2026-08-31';
-- type: ALL    = scan seluruh tabel (lambat)
-- type: range  = memakai index (cepat)
```

> Mengingat Pertemuan 4: strategi index dan `EXPLAIN`

Bagian 3

Memory Management

Kenapa Memory Bisa Habis

PHP punya batas `memory_limit` (umumnya 128 MB). Setiap row yang dimuat Eloquent menjadi sebuah **object model** lengkap dengan atribut dan metadata, jauh lebih besar daripada data mentahnya. Bayangkan meja kerja: `get()` menaruh semua dokumen di atas meja sekaligus. Begitu dokumennya ribuan, mejanya penuh dan pekerjaan berhenti.

> Contoh pengukuran di jobsheet: membaca 52.500 transaksi dengan `get()` memakai sekitar 96 MB, sedangkan `lazyById()` hanya sekitar 38 MB. Tambah datanya sedikit lagi, dan `get()` menabrak `memory_limit` : error *Allowed memory size exhausted*.

`get` vs `chunk` vs `lazy` vs `cursor`

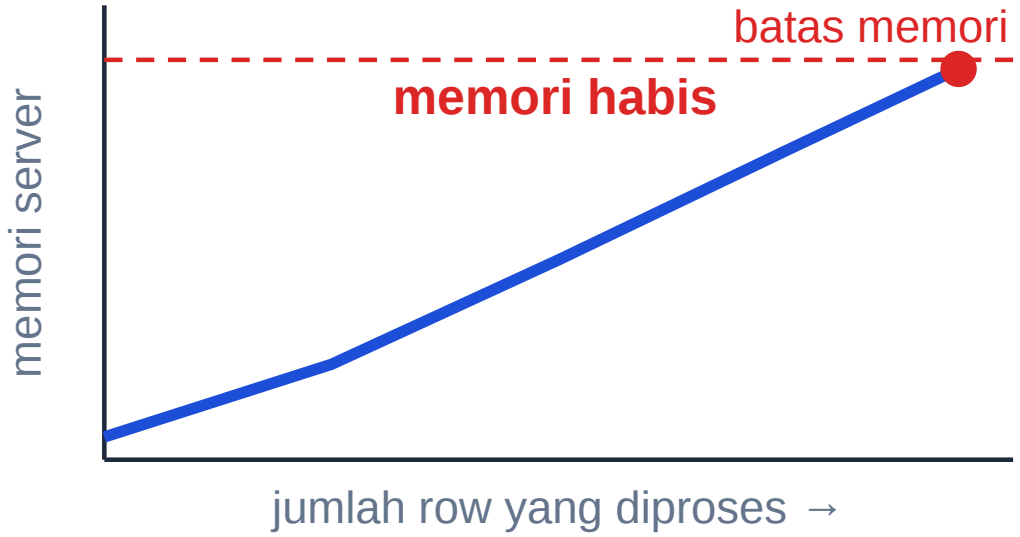
Cara	Memory	Query	Cocok untuk
`get()`	semua row sekaligus	1	data kecil, hasil ditampilkan
`chunkById(n)`	n row per giliran	banyak (per chunk)	update massal, proses per batch
`lazyById(n)`	n row per giliran	banyak (per chunk)	export, `foreach` biasa
`cursor()`	satu row per giliran	1	pembacaan sekali jalan, data tidak berubah

> Aturan praktis: kalau datanya bisa ratusan ribu row atau lebih, jangan `get()`. Pakai `lazyById()` untuk membaca dan `chunkById()` untuk meng-update.

Ember vs Keran: Dua Cara Mengirim File Besar

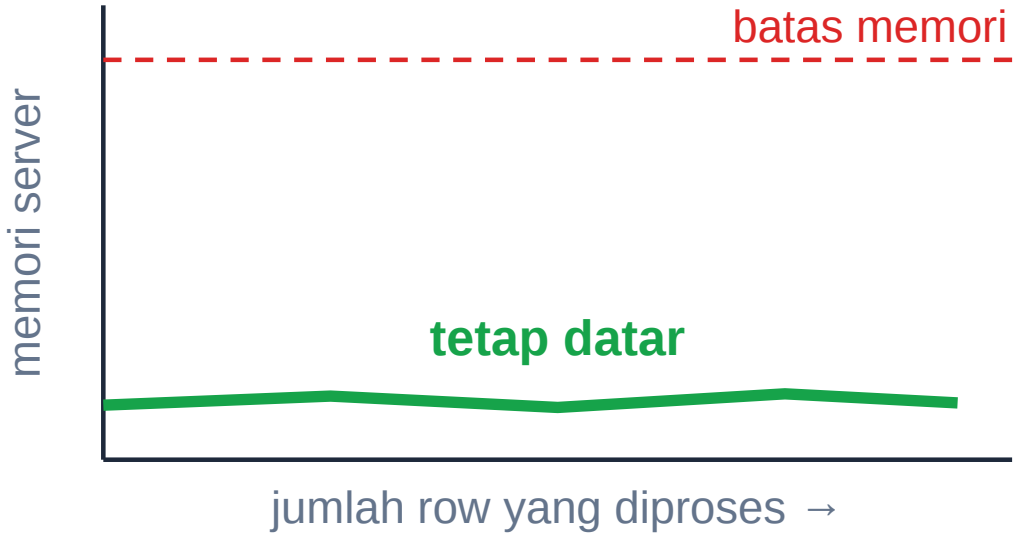
Ember: tampung dulu

get() lalu susun satu string besar



Keran: alirkan per row

lazyById() + streamDownload()



Export dengan `streamDownload()` dan `lazyById()`

Cara keran: ambil data per chunk kecil, tulis ke output, lalu lupakan.

```php
$articles = Article::with('author')
    ->whereBetween('published_at', [$from, $to]);

return response()->streamDownload(function () use ($articles) {
    $out = fopen('php://output', 'w');
    fputcsv($out, ['Judul', 'Penulis', 'Tanggal']);
    foreach ($articles->lazyById() as $a) {
        fputcsv($out, [$a->title, $a->author->name, $a->published_at]);
    }
    fclose($out);
}, 'laporan-artikel.csv');
```

`lazyById()` membaca database per chunk, dan `streamDownload()` mengirim tiap row sambil jalan, jadi memory tetap datar.

> Laravel docs: `HTTP Responses → File Downloads` dan `Eloquent → Chunking / lazyById()`

Jangan Menebak, Ukur

```php
gc_collect_cycles();
memory_reset_peak_usage();      // mulai hitung peak dari nol

foreach (Article::lazyById() as $a) {
    // ... pekerjaan ...
}

printf("peak %.1f MB\n", memory_get_peak_usage() / 1048576);
```

- `memory_get_peak_usage()` mencatat titik tertinggi pemakaian memory
- `memory_reset_peak_usage()` (PHP 8.2+) membuat tiap pengukuran berdiri sendiri
- Bandingkan dua cara pada data yang sama, dan catat angkanya

> Optimasi tanpa pengukuran hanyalah tebakan. Angka sebelum dan sesudah membuat keputusan "pakai `lazyById()`" bisa dipertanggungjawabkan.

Jebakan `chunk()`: Row yang Terlewat

```php
// BERBAHAYA: filter berdasarkan 'draft', lalu mengubah 'draft' itu sendiri
Article::where('status', 'draft')->chunk(100, function ($articles) {
    foreach ($articles as $a) {
        $a->update(['status' => 'published']);
    }
});
```

Setiap row yang di-update keluar dari filter, sehingga daftar bergeser dan chunk berikutnya melompati sebagian row. Seperti nomor urut di loket: kalau orang yang sudah dilayani langsung pergi, posisi orang di belakangnya ikut bergeser.

> Solusinya `chunkById(100, ...)`: ia memakai `id` terakhir sebagai penanda halaman, bukan nomor urut, sehingga aman walau row berubah.

Bagian 4

Format dan Library: CSV, Excel, PDF

Memilih Format: CSV, Excel, atau PDF

	CSV	Excel (.xlsx)	PDF
Cocok untuk	data mentah, pertukaran antar sistem	analisis, laporan keuangan	dibaca atau dicetak, tidak diolah lagi
Format	teks polos	zip berisi XML, ada styling	dokumen tetap
Memory	sangat hemat (stream)	sedang (library mengurus chunk)	berat (seluruh dokumen di memory)
Library	PHP bawaan (`fputcsv`)	Laravel Excel	DomPDF
Data besar	ya	ya (pakai `FromQuery`)	tidak, batasi row

CSV: Sederhana tapi Penuh Jebakan

- **Delimiter**: Excel versi Indonesia sering memakai titik koma (`;`), bukan koma
- **Quoting**: nilai yang mengandung koma atau baris baru harus diapit tanda kutip (`fputcsv` mengurusnya)
- **Encoding**: file UTF-8 dari Excel sering diawali karakter tak terlihat (**BOM**) yang merusak header pertama
- **Tanpa tipe**: semua nilai berupa teks, validasi tipe tetap tugasmu

> Kalau semua row tiba-tiba ditolak saat import, cek header-nya dulu: delimiter dan BOM adalah tersangka utama.

Laravel Excel: Export

Library spreadsheet mengurus format `.xlsx` yang rumit. Kamu cukup mendeskripsikan isinya lewat interface kecil (`concern`).

```
class ArticlesExport implements FromQuery, WithHeadings, WithMapping, ShouldAutoSize
{
    public function query()
    {
        return Article::with('author')->orderBy('id');
    }

    public function headings(): array { return ['Judul', 'Penulis', 'Tanggal']; }

    public function map($article): array
    {
        return [$article->title, $article->author->name, $article->published_at];
    }
}

return Excel::download(new ArticlesExport, 'artikel.xlsx');
```

> docs.laravel-excel.com → Exports

Laravel Excel: Import

```php
class ArticlesImport implements ToModel, WithHeadingRow, WithValidation,
    WithChunkReading, WithBatchInserts
{
    public function model(array $row): Article
    {
        return new Article(['title' => $row['title'], 'author_id' => $row['author_id']]);
    }

    public function rules(): array
    {
        return ['title' => ['required'], 'author_id' => ['exists:authors,id']];
    }

    public function chunkSize(): int { return 1000; }   // baca 1000 row per giliran
    public function batchSize(): int { return 500; }    // insert 500 row per query
}

Excel::import(new ArticlesImport, $request->file('file'));
```

Secara default, `WithValidation` menghentikan import pada row cacat pertama. Untuk melewati dan mengumpulkan row cacat, tambahkan `SkipsOnFailure`.

PDF dengan DomPDF: HTML Jadi Dokumen




```php
$pdf = Pdf::loadView('reports.articles', ['articles' => $articles])
    ->setPaper('a4', 'landscape');   // tabel lebar: pakai landscape

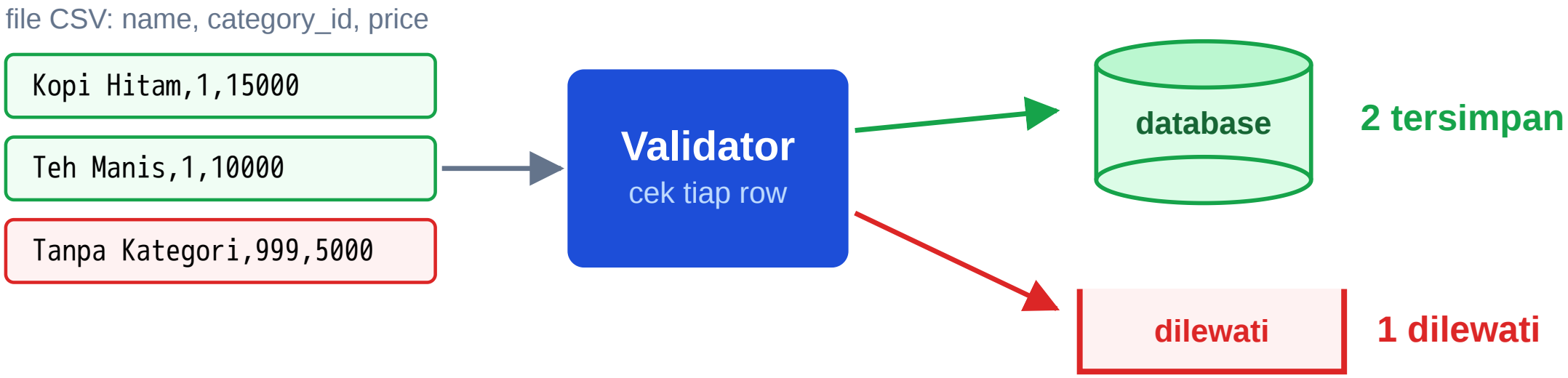
return $pdf->download('artikel.pdf');
```

- View-nya Blade biasa, jadi `@foreach`, `{{ }}`, dan CSS dasar langsung dipakai
- `download()` memaksa browser men-download file, `stream()` menampilkannya di browser

> DomPDF hanya mendukung sebagian CSS dan memuat seluruh dokumen di memory. Pakai untuk ringkasan dan tabel kecil, bukan untuk puluhan ribu row: untuk data besar pilih CSV atau Excel.

Import: Seperti Kasir yang Memindai Belanjaan

Import berjalan ke arah sebaliknya: membaca file CSV, lalu mengubah tiap row-nya jadi row baru di database. Bayangkan kasir memindai belanjaan. Kalau satu barcode rusak, kasir menyisihkan barang itu dan terus memindai sisanya, bukan menyuruh semua pelanggan pulang.



satu row cacat tidak menghentikan row lain

Import: Validasi Tiap Row, Lewati yang Cacat

```php
$rows = file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
$rows = array_map('str_getcsv', $rows);
$header = array_shift($rows);

foreach ($rows as $row) {
    $data = array_combine($header, $row);

    $validator = Validator::make($data, [
        'slug' => ['required', 'string'],
        'title' => ['required', 'string'],
        'author_id' => ['required', 'exists:authors,id'],
    ]);

    if ($validator->fails()) {
        continue; // lewati row ini, lanjut ke row berikutnya
    }

    Article::updateOrCreate(['slug' => $data['slug']], $validator->validated());
}
```

Apa yang Dilakukan dengan Row Cacat?

Strategi	Cara kerja	Cocok untuk
Skip tanpa jejak	row cacat dilewati tanpa catatan	hampir tidak pernah: data hilang tanpa ada yang tahu
Skip dan catat	row cacat dilewati, nomor row dan alasannya disimpan	import umum (disarankan): pengguna bisa memperbaiki file
Abort semua	satu row cacat membatalkan seluruh import (`DB::transaction`)	data keuangan atau data yang harus utuh

> Seperti kasir tadi: barang yang barcode-nya rusak disisihkan, **dan** dicatat di nota "tidak terbaca" supaya pelanggan tahu apa yang perlu diurus.

Bagian 5

Queue: Pekerjaan di Background

Dua Cara Memproses Pekerjaan

Langsung (dalam request)

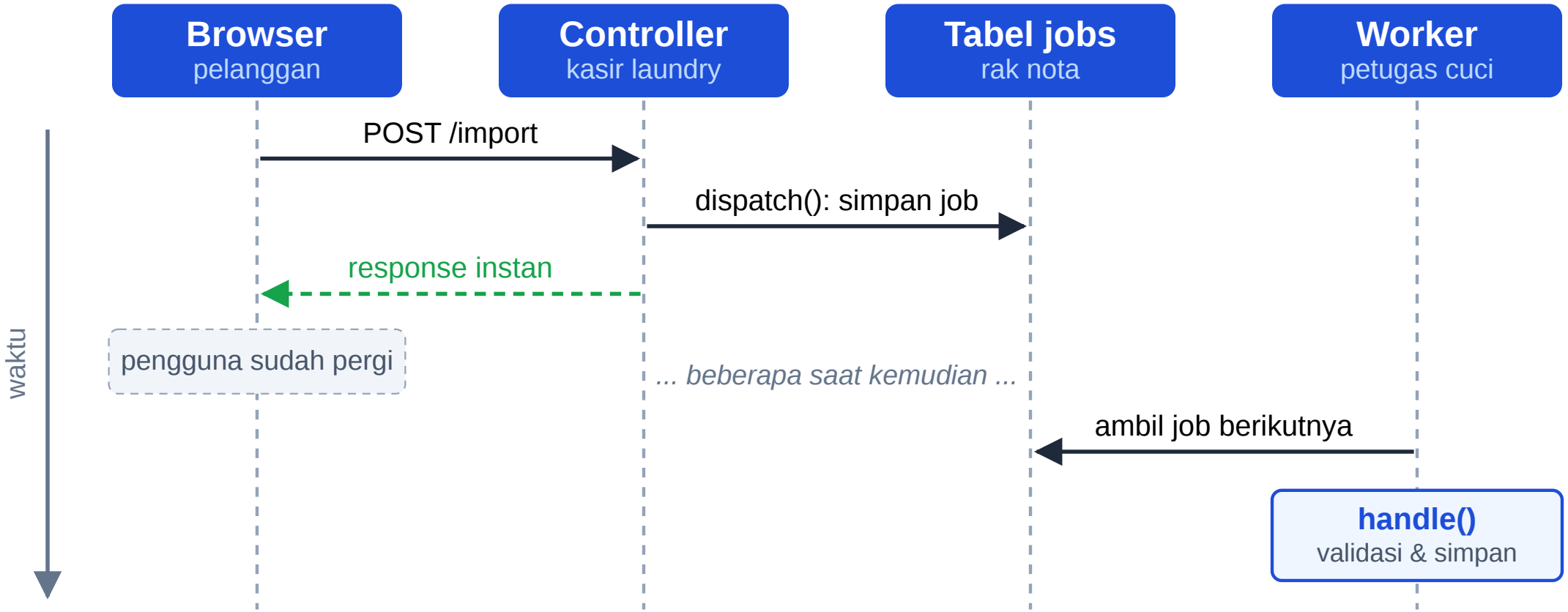
- Pengguna menunggu sampai selesai
- Kalau gagal, pengguna langsung tahu
- Cocok untuk pekerjaan singkat, hasilnya dibutuhkan seketika

Lewat Queue (job)

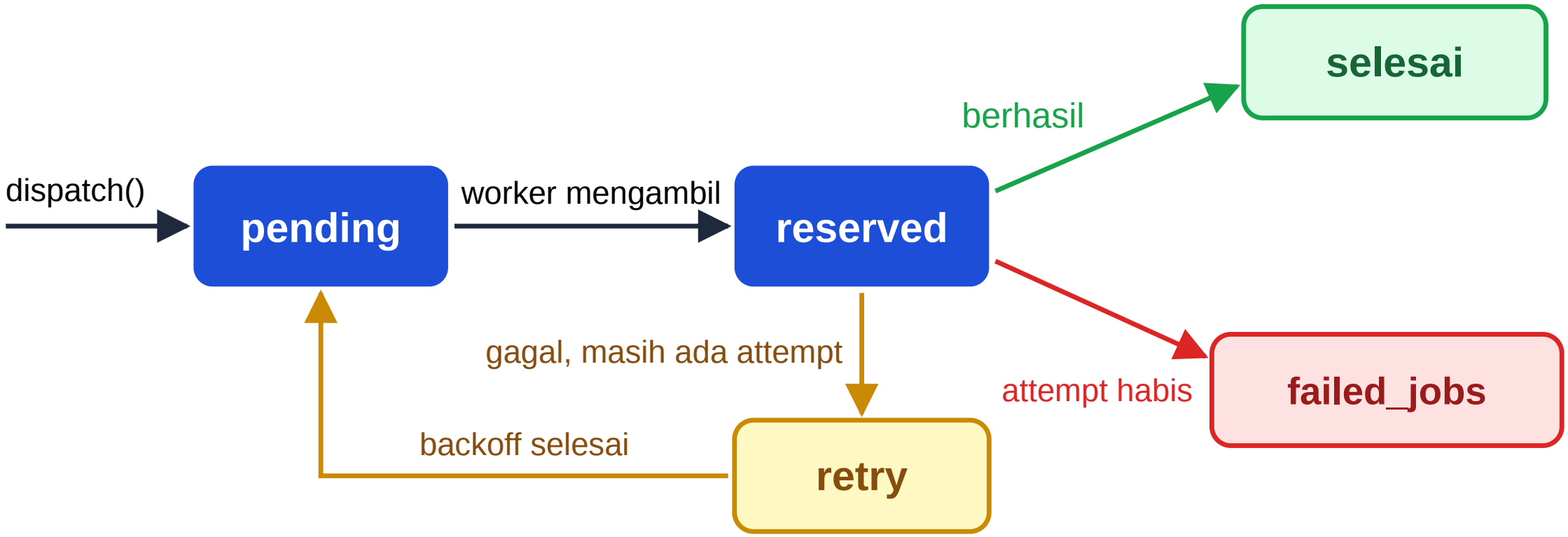
- Response instan, pekerjaan berjalan di background
- Pengguna tidak menunggu di depan browser
- Cocok untuk pekerjaan berat atau berdurasi tidak pasti

> Analogi laundry: ada cucian satu kaus, kamu rela menunggu di tempat. Ada cucian satu karung, kamu titip, dapat nota, lalu pulang. Import puluhan row itu satu kaus. Import ribuan row itu satu karung, dan HTTP request punya batas waktu tunggu.

Queue = Laundry Kiloan

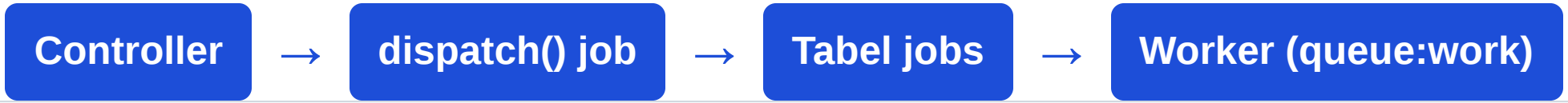


Perjalanan Sebuah Job




```
$tries = 3, $backoff = [10, 60]
```

Alur Job di Laravel




```
php artisan make:job ProcessArticleImport
```

```
ProcessArticleImport::dispatch($csvPath);

return back()->with('success', 'Import sedang diproses di background.');
```

Controller hanya menyimpan file dan mengirim satu job ke queue, lalu segera memberi response ke pengguna. Pekerjaan sesungguhnya (loop validasi dan penyimpanan) pindah ke metode `handle()` pada class job itu.

Worker: Petugas yang Mengerjakan Tumpukan Nota

> **Worker:** proses PHP terpisah yang terus berjalan, mengambil job dari tabel `jobs` satu per satu, lalu menjalankan metode `handle()` -nya.

```
php artisan queue:work
```

Tanpa worker, job hanya menumpuk di tabel `jobs`. Sama seperti laundry tanpa petugas cuci: nota terus bertambah, tapi tidak ada satu pun cucian yang selesai, karena `dispatch()` hanya menyimpan nota, bukan mencuci.

> Worker adalah proses yang hidup lama, jadi ia **tidak** otomatis memuat kode baru. Setelah mengubah class job, restart worker (di server production: `php artisan queue:restart`).

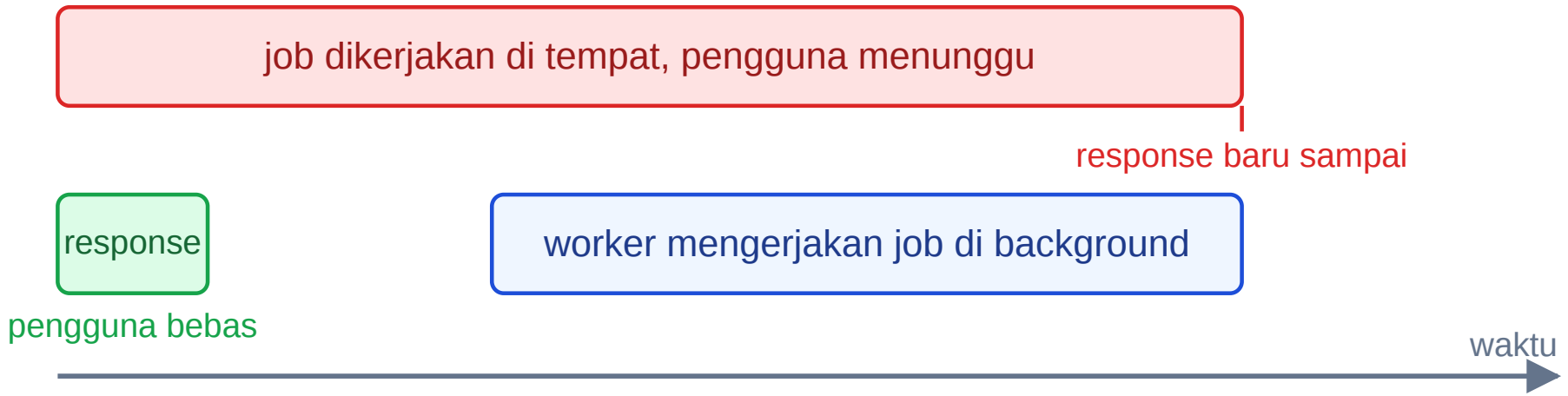
Driver Queue dan Worker di Production

Driver	Penyimpanan	Cocok untuk
`sync`	tidak ada: job dijalankan langsung di tempat	development dan testing
`database`	tabel `jobs`	proyek kecil sampai menengah, setup paling mudah
`redis`	Redis (in-memory)	trafik tinggi, butuh throughput besar

Di production, `queue:work` harus **terus hidup**. Program seperti **Supervisor** bertugas memantau worker dan menyalakannya kembali kalau mati, ibarat supervisor shift yang memastikan petugas cuci selalu ada di tempat.

> Laravel docs: `Queues → Supervisor Configuration`

`sync` vs `database`: Siapa yang Menunggu?



Kalau `QUEUE_CONNECTION` di `.env` bertuliskan `sync`, job tidak pernah masuk queue sungguhan. Ini seperti kasir yang menyuruhmu menunggu sampai cucian kering padahal kamu sudah memegang nota.

Job yang Tahan Gagal

Job berjalan di background, jadi tidak ada pengguna yang melihat error-nya. Karena itu job harus dirancang untuk gagal dengan rapi.

```
class ProcessArticleImport implements ShouldQueue
{
    use Queueable;

    public int $tries = 3;                // maksimal 3 attempt
    public array $backoff = [10, 60];     // jeda 10 detik, lalu 60 detik
    public int $timeout = 120;            // batalkan kalau lebih dari 120 detik
}
```

```
php artisan queue:failed          # lihat daftar job yang gagal
php artisan queue:retry all       # coba jalankan ulang semuanya
php artisan queue:flush           # hapus catatan job gagal
```

> Laravel docs: `Queues → Dealing With Failed Jobs`

Aturan Merancang Job

Lakukan

- Rancang job **idempotent**: aman dicoba ulang (`updateOrCreate`)
- Bawa ID atau path file, bukan data besar
- Atur `$tries`, `$backoff`, dan `$timeout`
- Gunakan `->afterCommit()` kalau job bergantung pada data di transaksi database

Hindari

- Memakai `request()`, session, atau user yang sedang login di dalam `handle()`: di background, semuanya tidak ada
- Menyimpan seluruh isi file di payload job
- Mengandalkan urutan job tanpa dirancang
- Membiarkan job gagal tanpa ada yang memantau

Memberi Kabar ke Pengguna

Pengguna yang meng-upload file lalu melihat "sedang diproses" ingin tahu kapan selesai. Pilihannya:

- **Column status** pada tabel import (`pending`, `processing`, `done`, `failed`) yang di-update oleh job, lalu ditampilkan di halaman
- **Notifikasi** (email atau notifikasi di aplikasi) saat job selesai atau gagal
- **Job batch**: `Bus::batch($jobs)->dispatch()` melacak banyak job sekaligus, dan `$batch->progress()` memberi persentase selesai

> Laravel docs: `Queues → Job Batching`

Tanda-Tanda Pekerjaan Layak Masuk Queue

- Durasinya tidak bisa diprediksi di muka, tergantung ukuran data yang diproses
- Hasilnya tidak perlu dilihat pengguna seketika di halaman yang sama
- Melibatkan panggilan ke layanan eksternal yang lambat, misalnya mengirim email atau memanggil API pihak ketiga
- Bisa dicoba ulang dengan aman (idempotent)

> Laporan yang dibuka dan dibaca langsung di halaman biasanya tetap lebih baik dikerjakan langsung, karena pengguna memang menunggu hasilnya saat itu juga. Queue bukan solusi untuk semua pekerjaan berat, hanya untuk yang tidak butuh hasil instan.

Menerapkan pada Simple POS

Konsep pada pertemuan ini akan kamu terapkan langsung pada Simple POS di jobsheet praktikum:

1. Laporan penjualan dengan filter tanggal, dihitung di database
2. Export laporan ke CSV (streaming), Excel (Laravel Excel), dan PDF (DomPDF)
3. Mengukur memory `get()` vs `lazyById()` pada 50.000 row
4. Import produk dari CSV yang validasi per row dan aman diulang
5. Memindahkan import ke queued job dengan `$tries` dan `backoff`, lalu menelusuri job yang gagal

> Kode lengkap: `github.com/se-polinema/simple-pos-ch08`

Rangkuman (1/3): ETL, Batch, dan Dataset Besar

- ETL memisahkan pengolahan data jadi extract, transform, load; import dan export adalah ETL dua arah
- Batch mengurangi round trip ke database (`array_chunk` + `insert`, `chunkById`); ukuran batch perlu diukur
- Import yang baik itu idempotent: `updateOrCreate()` atau `upsert()` (dengan unique index)
- Laporan merekap lewat `whereBetween`, `startOfDay()`/`endOfDay()` mencegah hari terakhir terpotong
- Hitung di database (`sum`, `groupBy`), jangan di PHP; index mempercepat filter

Rangkuman (2/3): Memory dan Format

- Memory habis karena `get()` memuat semua row sebagai object; pakai `lazyById()` untuk membaca dan `chunkById()` untuk meng-update
- `chunk()` bisa melewatkan row kalau column filter ikut di-update: pakai `chunkById()`
- Ukur dengan `memory_get_peak_usage()`, jangan menebak
- CSV untuk data mentah, Excel (Laravel Excel) untuk analisis, PDF (DomPDF) untuk dokumen ringkas
- Row cacat saat import: skip dan catat adalah pilihan umum, abort untuk data yang harus utuh

Rangkuman (3/3): Queue

- Pekerjaan berdurasi tidak terprediksi atau bervolume besar dititipkan ke queue: response instan, pekerjaan menyusul
- `dispatch()` hanya menyimpan job ke tabel `jobs`; worker (`queue:work`) yang menjalankannya
- Job tahan gagal: `$tries`, `$backoff`, `$timeout`, dan tabel `failed_jobs` dengan `queue:retry`
- Rancang job idempotent, bawa ID atau path (bukan data besar), jangan bergantung pada request atau session
- Di production, worker dijaga Supervisor dan di-restart saat deploy

Referensi & Diskusi

Dokumentasi resmi Laravel (Queues, File Storage, HTTP Responses, Eloquent Chunking), docs.laravel-excel.com, dan laravel-dompdf

Kode lengkap: `github.com/se-polinema/simple-pos-ch08`

Pertemuan berikutnya: Merancang dan Membangun REST API